

(continued)

Here's our original version:

```
def map2[A,B,C](a: Option[A], b: Option[B])(f: (A, B) => C):
  Option[C] =
    a flatMap (aa =>
      b map (bb =>
        f(aa, bb)))
```

And here's the *exact same code* written as a for-comprehension:

```
def map2[A,B,C](a: Option[A], b: Option[B])(f: (A, B) => C):
  Option[C] =
    for {
      aa <- a
      bb <- b
    } yield f(aa, bb)
```

A for-comprehension consists of a sequence of bindings, like `aa <- a`, followed by a `yield` after the closing brace, where the `yield` may make use of any of the values on the left side of any previous `<-` binding. The compiler desugars the bindings to `flatMap` calls, with the final binding and `yield` being converted to a call to `map`.

You should feel free to use for-comprehensions in place of explicit calls to `flatMap` and `map`.

Between `map`, `lift`, `sequence`, `traverse`, `map2`, `map3`, and so on, you should *never* have to modify any existing functions to work with optional values.

4.4 The Either data type

The big idea in this chapter is that we can represent failures and exceptions with ordinary values, and write functions that abstract out common patterns of error handling and recovery. `Option` isn't the only data type we could use for this purpose, and although it gets used frequently, it's rather simplistic. One thing you may have noticed with `Option` is that it doesn't tell us anything about what went wrong in the case of an exceptional condition. All it can do is give us `None`, indicating that there's no value to be had. But sometimes we want to know more. For example, we might want a `String` that gives more information, or if an exception was raised, we might want to know what that error actually was.

We can craft a data type that encodes whatever information we want about failures. Sometimes just knowing whether a failure occurred is sufficient, in which case we can use `Option`; other times we want more information. In this section, we'll walk through a simple extension to `Option`, the `Either` data type, which lets us track a *reason* for the failure. Let's look at its definition:

```
sealed trait Either[+E, +A]
case class Left[+E](value: E) extends Either[E, Nothing]
case class Right[+A](value: A) extends Either[Nothing, A]
```

Either has only two cases, just like `Option`. The essential difference is that both cases carry a value. The `Either` data type represents, in a very general way, values that can be one of two things. We can say that it's a *disjoint union* of two types. When we use it to indicate success or failure, by convention the `Right` constructor is reserved for the success case (a pun on “right,” meaning correct), and `Left` is used for failure. We've given the left type parameter the suggestive name `E` (for *error*).⁴

Option and Either in the standard library

As we mentioned earlier in this chapter, both `Option` and `Either` exist in the Scala standard library (`Option` API is at <http://mng.bz/fiJ5>; `Either` API is at <http://mng.bz/106L>), and most of the functions we've defined here in this chapter exist for the standard library versions.

You're encouraged to read through the API for `Option` and `Either` to understand the differences. There are a few missing functions, though, notably `sequence`, `traverse`, and `map2`. And `Either` doesn't define a right-biased `flatMap` directly like we do here. The standard library `Either` is slightly (but only slightly) more complicated. Read the API for details.

Let's look at the `mean` example again, this time returning a `String` in case of failure:

```
def mean(xs: IndexedSeq[Double]): Either[String, Double] =
  if (xs.isEmpty)
    Left("mean of empty list!")
  else
    Right(xs.sum / xs.length)
```

Sometimes we might want to include more information about the error, for example a stack trace showing the location of the error in the source code. In such cases we can simply return the exception in the `Left` side of an `Either`:

```
def safeDiv(x: Int, y: Int): Either[Exception, Int] =
  try Right(x / y)
  catch { case e: Exception => Left(e) }
```

As we did with `Option`, we can write a function, `Try`, which factors out this common pattern of converting thrown exceptions to values:

```
def Try[A](a: => A): Either[Exception, A] =
  try Right(a)
  catch { case e: Exception => Left(e) }
```

⁴ `Either` is also often used more generally to encode one of two possibilities in cases where it isn't worth defining a fresh data type. We'll see some examples of this throughout the book.

**EXERCISE 4.6**

Implement versions of `map`, `flatMap`, `orElse`, and `map2` on `Either` that operate on the `Right` value.

When mapping over the right side, we must promote the left type parameter to some supertype, to satisfy the `+E` variance annotation.

Similarly for `orElse`.

```
trait Either[+E, +A] {
  def map[B](f: A => B): Either[E, B]
  def flatMap[EE >: E, B](f: A => Either[EE, B]): Either[EE, B]
  def orElse[EE >: E, B >: A](b: => Either[EE, B]): Either[EE, B]
  def map2[EE >: E, B, C](b: Either[EE, B])(f: (A, B) => C):
    Either[EE, C]
}
```

Note that with these definitions, `Either` can now be used in for-comprehensions. For instance:

```
def parseInsuranceRateQuote(
  age: String,
  numberOfSpeedingTickets: String): Either[Exception, Double] =
  for {
    a <- Try { age.toInt }
    tickets <- Try { numberOfSpeedingTickets.toInt }
  } yield insuranceRateQuote(a, tickets)
```

Now we get information about the actual exception that occurred, rather than just getting back `None` in the event of a failure.

**EXERCISE 4.7**

Implement `sequence` and `traverse` for `Either`. These should return the first error that's encountered, if there is one.

```
def sequence[E, A](es: List[Either[E, A]]): Either[E, List[A]]

def traverse[E, A, B](as: List[A])(
  f: A => Either[E, B]): Either[E, List[B]]
```

As a final example, here's an application of `map2`, where the function `mkPerson` validates both the given name and the given age before constructing a valid `Person`.

Listing 4.4 Using `Either` to validate data

```
case class Person(name: Name, age: Age)
sealed class Name(val value: String)
sealed class Age(val value: Int)
```